Hippocampus Memory Architecture

Part of the are-self Technical Specification

are-self Project
`Independent Research`

March 2026

Abstract

This document specifies the memory architecture of the Hippocampus subsystem within Talos, the central cognitive engine. It proceeds in two main stages: (1) a formal description of the current pgvector-based implementation as it exists in production code, (2) the target directed hypergraph architecture as specified in the migration design document. All claims are epistemically tagged as **[Verified]**, **[Synthesized]**, or **[Speculative Proposal]** to maintain strict source-grounding throughout.

Contents

1 Introduction and Architectural Context **3**

2 Current Architecture: pgvector-Based Hippocampus **3**
2.1 Core Data Model: TalosEngram . . . 3
2.2 Embedding Function and Vector Space . . . 4
2.3 Similarity, Intercept, and Novelty . . . 4
2.4 Memory Operations (MCP Tool Interface) . . . 5
2.5 Catalog Injection Pattern . . . 6
2.6 Data Flow Summary . . . 6

3 Target Architecture: Typed Directed Hypergraph **6**
3.1 Core Hypergraph Definition . . . 6
3.2 Retrieval: Geodesic Traversal . . . 7
3.3 Discrete Novelty and Graph-Theoretic Deduplication . . . 7
3.4 Neo4j Compatibility: Bipartite Reduction Middleware . . . 8
3.5 Sleep Cycle Consolidation . . . 8
3.6 Planned Module Architecture . . . 9

4 The Continuous–Discrete Bridge **9**

5 Migration Strategy **10**
5.1 Legacy Data Migration . . . 10
5.2 Rollback Strategy . . . 10

6 Architectural Delta Summary **11**

7 Open Questions and Future Work **12**

8 References **12**

1 Introduction and Architectural Context

[Verified] The Hippocampus is one module within a biomimetic neural architecture that maps AI subsystems to brain regions. Talos decomposes into the following modules:

Module	Neural Analog	Responsibility
`central_nervous_system`	CNS	Labeled directed graph executor
`frontal_lobe`	Frontal Lobe	Reasoning loop (LLM sessions)
`parietal_lobe`	Parietal Lobe	Tool bridge (MCP gateway)
`hippocampus`	Hippocampus	Long-term memory (engrams)
`temporal_lobe`	Temporal Lobe	Scheduler / iteration state machine
`prefrontal_cortex`	Prefrontal Cortex	Work allocation (Agile tasks)
`identity`	Self-Model	Persona and governance
`hypothalamus`	Hypothalamus	Model routing (cosine-distance)
`occipital_lobe`	Occipital Lobe	Log digestion / error extraction
`thalamus`	Thalamus	Sensory / signaling layer
`synaptic_cleft`	Synaptic Cleft	WebSocket event bus

Table 1: Biomimetic module decomposition of Talos.

[Verified] The Hippocampus serves as Talos's durable long-term memory store. Its role is analogous to the biological hippocampus: crystallizing experiential facts into persistent *engrams*, screening them for redundancy, and providing retrieval pathways for downstream reasoning.

Remark 1.1. The architectural evolution described in this document proceeds along a single trajectory: from a purely continuous vector-embedding system (pgvector) to a typed directed hypergraph with continuous geometry retained as an auxiliary substrate. The fundamental shift is from fuzzy approximate retrieval to deterministic algorithmic traversal.

2 Current Architecture: pgvector-Based Hippocampus

[Verified] This section describes the Hippocampus as it exists in the production codebase. All definitions, formulas, and behaviors are drawn directly from `hippocampus/models.py`, `hippocampus/hippocampus.py`, and the auto-generated documentation in `docs/doc-generator/`.

2.1 Core Data Model: TalosEngram

[Verified] The fundamental unit of long-term memory is the **TalosEngram**, a Django model backed by PostgreSQL with the pgvector extension.

Definition 2.1 (TalosEngram)**.** A TalosEngram is a tuple:

$$\mathcal{E} = (\texttt{id}, \texttt{name}, \texttt{desc}, \mathbf{v}, r, \texttt{active}, \mathcal{L})$$

where:

- $\texttt{id} \in \text{UUID}$ — unique identifier (primary key)
- $\texttt{name} \in \Sigma^{\leq 254}$ — unique hash title
- $\texttt{desc} \in \Sigma^*$ — the actual fact or memory content
- $\mathbf{v} \in \mathbb{R}^{768}$ — embedding vector (pgvector `VectorField`)

- $r \in [0, 1]$ — relevance score
- `active` $\in \{\top, \bot\}$ — activation flag
- $\mathcal{L}$ — a set of relational links (detailed below)

[Verified] The relational links $\mathcal{L}$ connect each engram to the broader cognitive context:

Relation	Target Model	Cardinality
`sessions`	`ReasoningSession`	Many-to-Many
`source_turns`	`ReasoningTurn`	Many-to-Many
`spikes`	`Spike` (CNS graph nodes)	Many-to-Many
`tags`	`TalosEngramTag`	Many-to-Many
`tasks`	`PFCTask` (Agile work items)	Many-to-Many
`creator`	`IdentityDisc`	Foreign Key
`identity_discs`	`IdentityDisc`	Many-to-Many

Table 2: Relational links of TalosEngram.

Remark 2.1. Three distinct models in the codebase carry pgvector embeddings at 768 dimensions: `TalosEngram` (memory), `IdentityDisc` (persona), and `AIModel` (model catalog). All inherit from the shared `VectorMixin` abstract base class defined in `common/models.py`.

2.2 Embedding Function and Vector Space

[Verified] The embedding function maps text payloads into a 768-dimensional real vector space:

Definition 2.2 (Embedding Function)**.**

$$\phi : \mathcal{T} \longrightarrow \mathbb{R}^{768}$$

where $\mathcal{T}$ is the space of text strings and ϕ is realized by the `nomic-embed-text` model served via a local Ollama instance (accessed through `frontal_lobe.synapse.OllamaClient`).

The text payload is constructed as:

$$t = \texttt{"Title: "} \| \texttt{title} \| \texttt{"\textbackslash nFact: "} \| \texttt{fact}$$

where $\|$ denotes string concatenation.

2.3 Similarity, Intercept, and Novelty

[Verified] The Hippocampus uses cosine distance as its similarity metric, with a fixed intercept threshold for deduplication.

Definition 2.3 (Cosine Distance and Similarity)**.** For vectors $\mathbf{a}, \mathbf{b} \in \mathbb{R}^{768}$:

$$d_{\cos}(\mathbf{a}, \mathbf{b}) = 1 - \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \, \|\mathbf{b}\|} = 1 - \cos\theta$$

$$\mathrm{sim}(\mathbf{a}, \mathbf{b}) = 1 - d_{\cos}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \, \|\mathbf{b}\|}$$

Definition 2.4 (Intercept Condition — Auto-Recall Defense)**.** Given a candidate engram with embedding $\mathbf{v}_{\text{new}}$ and the set of all existing engram vectors $\{\mathbf{v}_i\}_{i=1}^{N}$:

$$\max_i \ \text{sim}(\mathbf{v}_{\text{new}}, \mathbf{v}_i) \geq 0.90 \quad \Longrightarrow \quad \text{Save } \textbf{rejected} \text{ (intercept)}$$

When intercepted, the system returns the existing engram reference rather than creating a duplicate. This serves as a deduplication defense.

Definition 2.5 (Novelty and Reward Functions)**.** For a non-intercepted save with maximum similarity s:

$$\text{novelty} = \max(0,\ 1 - s) \tag{1}$$

$$\text{focus_yield} = \max\big(1,\ \lfloor 10 \cdot \text{novelty} \rfloor\big) \tag{2}$$

$$\text{xp_yield} = \max\big(5,\ \lfloor 100 \cdot \text{novelty} \rfloor\big) \tag{3}$$

When the save is intercepted:

$$\text{focus_yield} = 0, \quad \text{xp_yield} = 0$$

Invariant 2.1 (Current System Invariants)**.** The following invariants hold in the production system:

1. **Dimensionality:** $\dim(\mathbf{v}) = 768$ for all stored vectors.
2. **Similarity range:** $\text{sim}(\mathbf{a}, \mathbf{b}) \in [0, 1]$.
3. **Intercept threshold:** Fixed at 0.90.
4. **Novelty bounds:** $\text{novelty} \in [0, 1]$.

2.4 Memory Operations (MCP Tool Interface)

[Verified] The Parietal Lobe exposes four MCP tools that serve as the LLM-facing API for memory operations. All invoke `TalosHippocampus` methods:

Tool	Behavior
`mcp_engram_save`	Embed title+fact → cosine scan → intercept or create. Links session, spike, turn, identity, tags. Returns `HippocampusMemoryYield`.
`mcp_engram_update`	Fetch by title → append `[UPDATE]: {fact}` to description → re-embed → update vector. Maintains all links.
`mcp_engram_read`	Fetch by UUID → attach session/spike/identity links → return formatted fact.
`mcp_engram_search`	PostgreSQL full-text search (`SearchVector`/`SearchQuery`) over name and description, with optional tag filter.

Table 3: Parietal MCP memory tools.

2.5 Catalog Injection Pattern

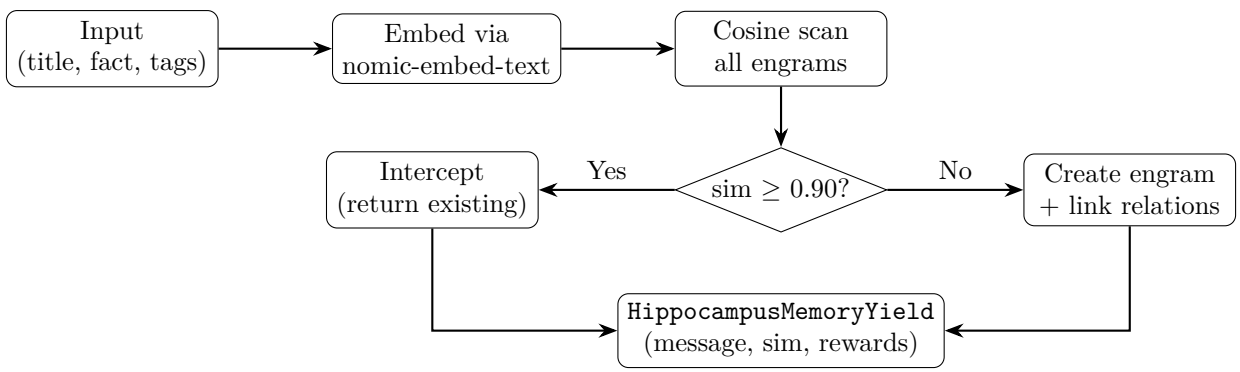
[Verified] The Hippocampus does not dump all memories into context. Instead, it uses a *catalog injection* pattern mediated by the `hippocampus_addon` (an identity addon):

- **Turn 1:** `get_turn_1_catalog(spike, limit=15)` — returns a catalog of prior engrams linked to the same `Spike.neuron`, injected as a volatile `ChatMessage`.
- **Turn $n > 1$:** `get_recent_catalog(session)` — returns engrams created or read during the current session.

Remark 2.2. This design separates *awareness* (knowing a memory exists via its catalog entry) from *recall* (explicitly reading the full content via `mcp_engram_read`). Catalog presence is cheap; full recall is deliberate.

2.6 Data Flow Summary

[Verified] The complete data flow for a memory save operation:



3 Target Architecture: Typed Directed Hypergraph

[Verified] This section draws from the migration design document. The target architecture reconceptualizes long-term memory as an exact hypergraph structure, replacing fuzzy vector retrieval with deterministic algorithmic traversal.

3.1 Core Hypergraph Definition

Definition 3.1 (Directed Hypergraph)**.** The new Hippocampus is modeled as a directed hypergraph:

$$H = (V, E)$$

where:

- V is a finite set of **vertices** representing discrete contextual system states.
- $E \subseteq \bigcup_{k \geq 2} V^k$ is a set of **directed hyperedges** representing relations or actions linking multiple elements.

Definition 3.2 (Ternary Engram Hyperedge)**.** An operational memory (engram) is modeled as a ternary relation (arity $k = 3$):

$$e_i = (v_{\text{state}},\ v_{\text{action}},\ v_{\text{next_state}})$$

where:

- $v_{\text{state}} \in V$ — the system state at the time of memory formation.
- $v_{\text{action}} \in V$ — the action or event that occurred.
- $v_{\text{next_state}} \in V$ — the resulting system state.

Higher-order contexts or temporal links correspond to increasing the arity of the hyperedge beyond $k = 3$.

Remark 3.1. This ternary structure (s, a, s') is isomorphic to the standard state–action–next-state transition tuple used in reinforcement learning. The hypergraph formulation generalizes this by allowing a single hyperedge to connect arbitrarily many vertices simultaneously.

3.2 Retrieval: Geodesic Traversal

[Verified] Memory retrieval transitions from approximate nearest-neighbor search to exact graph traversal:

Definition 3.3 (Geodesic Distance)**.** The distance between two vertices $v_s, v_t \in V$ is defined as the number of hyperedges on the shortest path between them:

$$d_H(v_s, v_t) = \min \{|P| : P \text{ is a path from } v_s \text{ to } v_t \text{ via hyperedges in } E\}$$

This replaces cosine distance as the fundamental retrieval metric.

1: Memory Retrieval via Geodesic Traversal

```
proc RetrieveEngram(current_state, target_goal):
    v_s ← FindNode(current_state)
    v_t ← FindNode(target_goal)
    P ← FindGeodesic(H, v_s, v_t) # Shortest path via hyperedges
    return P # Exact logical pathway
end proc
```

3.3 Discrete Novelty and Graph-Theoretic Deduplication

[Verified] The novelty computation must be redefined for the discrete setting:

Definition 3.4 (Discrete Novelty)**.** When saving a new engram e_{new}, the system checks for isomorphic sub-hypergraphs within H:

- If an exact isomorphic match exists: sim = 1.0 (intercept).
- Otherwise: sim scales inversely with the *graph edit distance* between e_{new} and the nearest existing subgraph.
- Novelty scales by how much the new hyperedge increases the *topological volume* of the local neighborhood.

2: Memory Addition via Hypergraph Rewriting

proc AddEngram(state, action, next_state):
  $v_c \leftarrow$ GetOrCreateNode(state)
  $v_n \leftarrow$ GetOrCreateNode(next_state)
  $e_{\text{new}} \leftarrow$ CreateDirectedHyperedge$(v_c, \text{action}, v_n)$
  sim $\leftarrow$ CheckIsomorphism(H, e_{new})
  if sim ≥ 0.90:
  return Rejected # *Legacy intercept threshold preserved*
  end if
  RewriteGraph(H, e_{new}) # *Apply hypergraph rewrite rule*
  CalculateAndGrantRewards(sim)
end proc

3.4 Neo4j Compatibility: Bipartite Reduction Middleware

[Verified] A fundamental tension exists between the hypergraph formalism (which requires k-ary relations) and Neo4j (which supports only binary property-graph edges).

Definition 3.5 (Bipartite Reduction)**.** Each k-ary hyperedge $e \in E$ is projected into the Neo4j property graph by representing it as a distinct **hyperedge node**, connected via binary edges to its k constituent element nodes:

$$e = (v_1, v_2, \ldots, v_k) \quad \xrightarrow{\text{project}} \quad n_e \text{ (hyperedge node)}, \ \{(n_e \to v_i)\}_{i=1}^{k}$$

This standard bipartite reduction preserves the mathematical integrity of the hypergraph while adhering to Neo4j's storage constraints.

Remark 3.2. An alternative is to use NetworkX exclusively, which natively supports arbitrary data structures and avoids the bipartite reduction overhead. The trade-off is loss of Neo4j's production-grade persistence, indexing, and query optimization.

3.5 Sleep Cycle Consolidation

[Verified] The temporal lobe implements a cron-based "Sleep State" for memory consolidation:

3: Sleep Cycle Consolidation

proc SleepStateConsolidation():
  $\mathcal{E}_{\text{day}} \leftarrow$ GetDailyEngrams(H)
  for $e \in \mathcal{E}_{\text{day}}$:
  $\Delta \leftarrow$ ExtractImprovements(e) # *Identify consolidation targets*
  end for
  GenerateTrainingData(Δ)
end proc

[Verified] The Sleep State is located in `temporal_lobe/inception.py` and `tasks.py`. During low user activity, Talos suspends the frontal lobe, processes the day's engrams, and uses `inception.py` to simulate "dreams" (sandboxed synthetic scenarios).

3.6 Planned Module Architecture

[Verified] The migration document specifies three new core modules:

Module	Responsibility
`HypergraphMemoryManager`	Manages the bipartite Neo4j or NetworkX mapping. Handles vertex creation, hyperedge construction, and graph queries.
`EngramRewriter`	Implements hypergraph rewriting rules. The hypergraph evolves by transforming discrete collections of relations at each timestep.
`SleepConsolidator`	Generates synthetic `IterationDefinitions` during the sleep cycle. Implemented within `inception.py`.

Table 4: New core modules for the hypergraph Hippocampus.

4 The Continuous–Discrete Bridge

[Verified] The central unresolved tension in the migration is the mapping from continuous $\mathbb{R}^{768}$ embeddings to discrete hypergraph vertices:

> *"The exact mapping from $\mathbb{R}^{768}$ continuous embeddings to discrete hypergraph vertices remains an open question."*

[Verified] Four resolution approaches have been proposed:

1. **Hybrid Nodes** — Each vertex carries both a discrete UUID (for graph traversal) and a compressed vector (for fuzzy recall as fallback).
2. **Product Quantization** — FAISS-based 8–32× compression of the embedding vectors, preserving approximate similarity while reducing the continuous space to a manageable finite codebook.
3. **Learned Discretization (VQ-VAE)** — Train a codebook via Vector Quantized Variational Autoencoder:
$$\text{encoder} : \mathbb{R}^{768} \longrightarrow \{0, 1, \ldots, 4095\}$$
Each engram is represented by its codebook index rather than its full embedding vector.
4. **Hierarchical Decomposition** — Three nested levels:
   - Level 1: Discrete state nodes (vertices in V)
   - Level 2: Vector embeddings (auxiliary cached views)
   - Level 3: Hyperedge relations (edges in E)

Open Question 4.1. Which discretization strategy best preserves retrieval fidelity while enabling deterministic graph traversal? The VQ-VAE approach aligns well, but requires a sufficiently large and well-structured training corpus of engrams.

5 Migration Strategy

[Verified] The migration from pgvector to hypergraph proceeds in four phases:

Phase 1: Infrastructure — Deploy Neo4j (or initialize NetworkX in-process) alongside existing PostgreSQL and Redis.

Phase 2: Dual-Write — `mcp_engram_save` writes to both pgvector (semantic vectors) and the hypergraph (rewrite rule insertions). Both stores are kept in sync.

Phase 3: Cut-Over — Point `mcp_engram_read` and `mcp_engram_search` to hypergraph geodesic traversal endpoints. pgvector becomes read-only fallback.

Phase 4: Consolidation — Enable `inception.py` to process graph engrams during the sleep cycle. Full system operates on hypergraph as primary substrate.

5.1 Legacy Data Migration

[Verified] Existing 768-dimensional vectors are instantiated as disconnected vertices in the new hypergraph. A batch process constructs edges by evaluating cosine similarities as semantic links, bootstrapping the initial graph topology from the continuous metric.

5.2 Rollback Strategy

[Verified] The legacy pgvector repository is maintained as a read-only fallback. If hypergraph retrieval fails or produces degenerate paths, the system gracefully degrades to nearest-neighbor vector search.

6 Architectural Delta Summary

Aspect	**Current (pgvector)**	**Target (Hypergraph)**
Storage backend	PostgreSQL + pgvector	Neo4j or NetworkX + PostgreSQL (fallback)
Memory unit	768-dim vector + metadata	Ternary hyperedge (s, a, s')
Retrieval metric	Cosine similarity	Geodesic distance (shortest path)
Novelty computation	$\max(0, 1 - \text{sim})$	Isomorphism check + graph edit distance
Deduplication	Cosine threshold ≥ 0.90	Subgraph isomorphism detection
Consolidation	Not yet implemented	Sleep cycle via `inception.py`
Self-improvement	Weight Updates	Weight Updates + Future Structural Rewriting
Formal substrate	$\mathbb{R}^{768}$ vector space	Three-layered Hypergraph Substrate

Table 5: Architectural delta: current vs. target Hippocampus.

7 Open Questions and Future Work

Open Question 7.1. **Discretization strategy selection.** Which of the four proposed approaches (Hybrid Nodes, Product Quantization, VQ-VAE, Hierarchical Decomposition) should be implemented? The current architectural preference is for VQ-VAE with end-to-end training, storing codebook indices rather than raw vectors. **[Speculative Proposal]**

Open Question 7.2. **Intercept threshold adaptation.** The fixed 0.90 cosine threshold works for continuous embeddings but may need recalibration for discrete graph-theoretic novelty. Should the threshold be adaptive, learned, or replaced by a structural criterion (e.g., subgraph diameter)? **[Speculative Proposal]**

8 References

[1] **Talos Codebase** — `hippocampus/models.py`, `hippocampus/hippocampus.py`, `common/models.py`. Production source code. **[Verified]**

[2] **Hippocampus Documentation** — `docs/doc-generator_talos-only/hippocampus__documentation.md` Auto-generated documentation with mathematical appendix. **[Verified]**

[3] **LIVE_CODEBASE_ANALYSIS.md** — `docs/LIVE_CODEBASE_ANALYSIS.md`. System-wide analysis including memory system section. **[Verified]**

[4] **Hippocampus Memory Migration to Hypergraphs** — `docs/final_docs/are-self_hippocmpus_memor` Migration design document with algorithms and data model. **[Verified]**

[5] **are-self Whitepaper** — `docs/are_self_whitepaper.tex`. System design overview and formal definitions. **[Verified]**

[6] **are-self Preprint Article** — `docs/final_docs/are-self_preprint_article.pdf`. Brain-themed modular decomposition. **[Verified]**

[7] C. Berge, *Graphs and Hypergraphs*, North-Holland, 1973.

[8] A. Bretto, *Hypergraph Theory: An Introduction*, Springer, 2013.

[9] Y. Feng et al., "Hypergraph Neural Networks," *Proc. AAAI*, 2019.